ACI COMPANION READER · AIMLCZG557

Search Algorithms

Uninformed Search (BFS, DFS, UCS, IDS) and Informed Search (Greedy, A*)

Session 3 · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. The slides give you the formulas; this reader gives you the *why*. Every slide concept is expanded with a full plain-English explanation. Every slide example is worked out step by step with real numbers. Five colour-coded boxes guide you through each concept. The **blue** "intuition" box states the core idea in plain words. The **amber** "everyday picture" box gives a real-world analogy before any math. The **green** "worked example" box shows every step with no skipped lines. The **red** "watch out" box flags the common mistake. The **purple** "key takeaway" box is the one sentence to remember.

1 What this session covers

This session has two halves. The first half recaps the four uninformed search algorithms from last week: BFS, DFS, UCS, and IDS. The second half introduces informed (heuristic) search, where the agent uses a guess of the distance left to reach the goal. We cover two informed algorithms: Greedy Best First Search and A*. We end with the conditions that make A* optimal: admissibility and consistency.

Algorithm	Type	Uses heuristic?
Breadth First Search (BFS)	Uninformed	No
Depth First Search (DFS)	Uninformed	No
Uniform Cost Search (UCS)	Uninformed	No
Iterative Deepening Search (IDS)	Uninformed	No
Greedy Best First Search	Informed	Yes ($h(n)$)
A* Search	Informed	Yes ($g(n) + h(n)$)

2 Breadth First Search (BFS)

You are looking for a lost cat in a building. You check every room on the ground floor first, then every room on the first floor, then the second. You never go deeper until the current level is fully checked. That is exactly how BFS works.

BFS explores a search tree *level by level*. It keeps a **queue** (first-in, first-out) of nodes to visit. When a node is expanded, all its children join the back of the queue. So depth-1 nodes are all visited before any depth-2 node, and so on.

☞ **The intuition**

BFS is "cautious." It checks all short paths before any long path. The queue makes sure you never jump ahead: nodes further from the start wait their turn until everything at the current depth is done.

★ Everyday picture

Think of ripples on a pond. Drop a stone at the start node. The first ring of ripples reaches depth-1 nodes. The second ring reaches depth-2 nodes. BFS traces exactly those rings, one ring at a time, until it hits the goal.

Properties of BFS

- **Complete:** Yes. If the shallowest goal node is at depth d, BFS will find it. It generates all shallower nodes first, so it cannot miss a node at depth d.
- **Optimal:** Yes, if every action has the same cost. In that case the shallowest goal is also the cheapest. If costs vary, BFS is not optimal.
- **Time complexity:** $\mathcal{O}(b^d)$, where b is the **branching factor** (children per node) and d is the depth of the shallowest goal. At depth 1 there are b nodes, at depth 2 there are b^2, and at depth d there are b^d.
- **Space complexity:** $\mathcal{O}(b^d)$. The frontier (queue) holds at most all nodes at the current depth, which is b^d.

✗ Watch out

BFS is memory-hungry. For $b = 10$ and $d = 10$, the frontier holds 10^{10} nodes. That is ten billion nodes in memory. Use BFS only when d is small.

Where this shows up in ML. BFS is the foundation of many ML graph algorithms. It finds connected components in feature graphs. It computes multi-hop neighbourhoods in graph neural networks. It also does shortest-path preprocessing for knowledge-graph embeddings.

✓ Key takeaway

BFS visits nodes level by level using a queue. It is complete and optimal (equal costs), but its memory cost grows exponentially with depth.

3 Depth First Search (DFS)

Now imagine looking for the cat differently. You pick one corridor and follow it all the way to the end before trying any other corridor. You go as deep as you can, then backtrack. That is DFS.

DFS explores one full path from start to leaf before trying any sibling path. It keeps a **stack** (last-in, first-out) of nodes to visit. When a node is expanded, its children are pushed onto the stack. So the algorithm immediately dives into the first child, then into the first grandchild, and so on.

☞ The intuition

DFS is "aggressive." It chases one path as far as possible. Only when that path ends (a leaf or a dead end) does it backtrack and try the next branch. A stack naturally gives you this behaviour: the most recently added node is the first to be visited.

★ Everyday picture

You are reading a book with footnotes that have their own footnotes. DFS reads each footnote completely (including its sub-footnotes, and their sub-footnotes) before returning to the main text. BFS would read one footnote, then another, never nesting deeply.

Properties of DFS

- **Complete:** Yes, in finite state spaces. It will eventually expand every node.
- **Optimal:** No. DFS stops when it finds *any* goal, not the shallowest or cheapest one.
- **Time complexity:** $\mathcal{O}(b^m)$, where m is the maximum depth of any node in the tree. This can be much larger than d (the shallowest goal depth).
- **Space complexity:** $\mathcal{O}(bm)$. DFS only stores one path from root to a leaf at a time, plus its unexpanded siblings. Once a node is expanded and all its children visited, it can be removed from memory.

Where this shows up in ML. DFS underlies topological sorting of computation graphs (used in automatic differentiation frameworks like PyTorch), and cycle detection in data pipeline DAGs.

✓ Key takeaway

DFS uses a stack and dives deep first. It is complete in finite spaces and uses very little memory ($\mathcal{O}(bm)$), but is not optimal.

4 Uniform Cost Search (UCS): Cheapest First

Sometimes not all actions cost the same. A road from city A to city B might cost 80 km while another costs 140 km. BFS ignores those numbers. UCS does not: it always expands the node with the *lowest total path cost* so far.

UCS sorts the frontier as a **priority queue** ordered by the path cost $g(n)$, where $g(n)$ is the total cost of reaching node n from the start. It expands the cheapest node first, not the shallowest one.

☞ The intuition

UCS is BFS but with costs. Instead of asking "how many steps to here?" it asks "how much did it cost to get here?" The priority queue ensures you always try the cheapest partial path next.

★ Everyday picture

You are navigating a map and each road has a toll. You want the cheapest route to Bucharest. You would not just take the fewest roads; you would take the roads that add up to the lowest total toll. UCS does exactly that: it keeps a running tab and always extends the tab that is lowest so far.

The Sibiu-to-Bucharest example

The slides walk through UCS on a small graph. The cities and road costs are:

- Sibiu to Rimnicu Vilcea: 80
- Sibiu to Fagaras: 99
- Rimnicu Vilcea to Pitesti: 97
- Fagaras to Bucharest: 211
- Pitesti to Bucharest: 101

✎ Worked example: UCS from Sibiu to Bucharest

We want the cheapest path from Sibiu to Bucharest.

Step 1. **Start.** Expand Sibiu. Generate children: (*Rimnicu Vilcea*, cost 80) and (*Fagaras*, cost 99). Frontier: {Rimnicu Vilcea:80, Fagaras:99}.

Step 2. **Expand cheapest: Rimnicu Vilcea (cost 80).** Generate child: (*Pitesti*, cost $80 + 97 = 177$). Frontier: {Fagaras:99, Pitesti:177}.

Step 3. **Expand cheapest: Fagaras (cost 99).** Generate child: (*Bucharest*, cost $99 + 211 = 310$). This is the goal, but we do *not* test during generation — we add to frontier. Frontier: {Pitesti:177, Bucharest:310}.

Step 4. **Expand cheapest: Pitesti (cost 177).** Generate child: (*Bucharest*, cost $177 + 101 = 278$). This is cheaper than the existing Bucharest entry (310), so we replace it. Frontier: {Bucharest:278}.

Step 5. **Expand cheapest: Bucharest (cost 278).** Goal test satisfied. Return the path Sibiu → Rimnicu Vilcea → Pitesti → Bucharest with total cost **278**.

The path via Fagaras (cost 310) was discarded when Pitesti gave us a cheaper route to Bucharest. This is why we test *on expansion*, not on generation.

✗ Watch out

Do not apply the goal test when a node is first *generated*. Apply it when the node is *expanded* (i.e. taken off the priority queue). If you test on generation, you may accept a sub-optimal path to the goal: a cheaper route might arrive later in the queue.

Where this shows up in ML. UCS is Dijkstra's algorithm with a different framing. It is used in knowledge-graph shortest paths, token-level beam decoding costs, and computing minimum edit distances in sequence-to-sequence models.

✓ Key takeaway

UCS expands the cheapest node first. It is complete and optimal for non-negative costs. Always apply the goal test on expansion, not generation.

5 Iterative Deepening Search (IDS)

BFS is optimal but eats memory. DFS is memory-efficient but not optimal. IDS gets the best of both by running DFS repeatedly with a growing depth limit.

Iterative Deepening Search (IDS) runs a **Depth Limited Search (DLS)** starting with limit $l = 0$. DLS is DFS with a cutoff: nodes deeper than l are not expanded. If no goal is found, IDS increases l by one and restarts. It keeps going until a goal is found.

☞ The intuition

IDS is "trying on progressively larger boxes." Start with a very small box (limit 0). If the goal does not fit, try the next larger box (limit 1), and so on. Each box is a full DFS, so memory stays low. Because you try all boxes in order, you never skip the shallowest goal.

★ Everyday picture

You are searching a city for your friend. You first check every location within 1 km (limit 1). Not found. Then every location within 2 km (limit 2). Not found. Then 3 km, and so on. You re-check close locations on each round, but that is fine because there are few of them.

The algorithm

```
function IDS(problem):
  for depth ← 0 to ∞ do
    result ← DLS(problem, depth)
    if result ≠ cutoff then return result
```

Properties of IDS

- **Complete:** Yes, when the branching factor is finite.
- **Optimal:** Yes, if all step costs are equal ($= 1$). It can be extended to find cost-optimal solutions by using cost thresholds (iterative lengthening) instead of depth limits.
- **Time complexity:** $\mathcal{O}(b^d)$, same order as BFS. The re-expansion of upper levels costs $(d+1)b^0 + d\,b^1 + (d-1)b^2 + \cdots + b^d$, which is $\mathcal{O}(b^d)$.
- **Space complexity:** $\mathcal{O}(bd)$, same as DFS. Only one path from root to the current leaf is stored.

✎ Worked example: IDS on a depth-3 tree

Suppose our tree has branching factor $b = 2$ and the goal is at depth $d = 3$.

Step 1. **Limit 0.** DLS explores only the root node. No goal. Cutoff.

Step 2. **Limit 1.** DLS explores root, then its two children (A and B). No goal. Cutoff.

Step 3. **Limit 2.** DLS dives to depth 2 under every branch. Explores root, A, A's children, B, B's children. No goal. Cutoff.

Step 4. **Limit 3.** DLS reaches depth 3. Suppose the goal is the first leaf visited. Return success.

Total nodes re-expanded at levels 0–2 add a constant overhead. For $b = 2$, $d = 3$: overhead is about $3\times$ the top levels, which is tiny compared to $2^3 = 8$ leaves at the goal level. So IDS costs roughly the same as BFS.

✗ Watch out

IDS re-expands upper-level nodes on every iteration. For $b = 10$ and $d = 5$, the root is expanded 6 times, depth-1 nodes 5 times, etc. This looks wasteful, but the total extra work is only a small fraction ($\approx b/(b-1)$ times) of the leaf-level work. For $b = 10$, re-expansion

adds only about 11% extra work.

Where this shows up in ML. IDS is the basis of **iterative deepening A*** (IDA*), used in game-tree search (chess endgame solvers) and theorem provers. The depth-limit idea also appears in beam-search decoding: a beam width is a kind of breadth limit that keeps memory small while still finding good sequences.

✓ **Key takeaway**

IDS combines BFS's optimality with DFS's small memory. It restarts DFS with an increasing depth limit. Time is $\mathcal{O}(b^d)$; space is $\mathcal{O}(bd)$.

6 From Uninformed to Informed Search

BFS, DFS, UCS, and IDS are all **uninformed** (also called *blind*) search algorithms. They have no idea whether one unexplored node is closer to the goal than another. They simply follow the rules of the data structure (queue, stack, priority queue) with no extra knowledge.

Informed search adds one ingredient: a **heuristic function** $h(n)$. This function estimates how far node n is from the goal. Armed with this estimate, the algorithm can focus attention on nodes that look promising and skip over nodes that look far away.

☞ **The intuition**

Uninformed search is like driving to a new city with no map — you try every road. Informed search is like having a GPS that estimates the remaining distance. You do not know the exact route, but you can tell which direction is roughly correct and go that way first.

Property	Uninformed	Informed
Uses domain knowledge	No	Yes ($h(n)$)
Speed	Slower	Faster
Always complete	Yes	Depends on $h(n)$
Always optimal	Depends on algorithm	Depends on $h(n)$
Examples	BFS, DFS, UCS, IDS	Greedy Best First, A*

A **heuristic function** $h(n)$ takes the current state and returns a non-negative number — the estimated cost to reach the goal from n. It is not guaranteed to be exact. It is a guess. A good guess speeds up the search; a bad guess can mislead it.

★ **Everyday picture**

You are navigating a city. You do not know the exact road distances, but you do know the straight-line (as-the-crow-flies) distance from each intersection to your destination. That straight-line distance is a heuristic. It is always an underestimate (you cannot travel in a straight line through buildings), but it points you in the right direction.

Where this shows up in ML. Heuristics appear in neural architecture search, where a learned score estimates how good a partial architecture is. They appear in program synthesis to judge how close a partial program is to correct. They also appear in LLM agent planning: a learned

value function scores each plan fragment.

7 Greedy Best First Search

The simplest informed search is **Greedy Best First Search (GBFS)**. At every step, it picks the node that looks *closest* to the goal, ignoring how far it has already traveled. It uses the evaluation function

$$f(n) = h(n),$$

where $h(n)$ is the heuristic estimate from n to the goal. The word "greedy" means the algorithm only cares about the next step, not the journey so far.

☞ The intuition

GBFS is like a compass. It always points toward the goal. It does not look back at how far you have walked; it just says "the goal looks that way." This is fast when the compass is right. But a compass can point you through a swamp even if there is a dry path around it.

Properties of GBFS

- **Not optimal.** The algorithm only optimises for the current step. A greedy choice can lead down a long path while a slightly worse first step would have reached the goal quickly.
- **Not complete.** GBFS can enter an infinite loop if the heuristic keeps pointing toward a dead end and the algorithm never backtracks past it.
- **Time and space complexity:** $\mathcal{O}(b^m)$, where m is the max depth. A good heuristic can dramatically reduce this in practice.

The Romania worked example (GBFS)

The slide uses the Romania road-map problem. Start: Arad; Goal: Bucharest. The heuristic $h(n)$ is the straight-line distance from each city to Bucharest (given in the table on slide 31). Key values: Arad 366, Sibiu 253, Timisoara 329, Zerind 374, Fagaras 176, Bucharest 0.

✎ Worked example: Greedy Best First: Arad to Bucharest

Step 1. **Start at Arad.** h(Arad) = 366. Expand Arad. Frontier: {Sibiu:253, Timisoara:329, Zerind:374}.

Step 2. **Expand Sibiu** (lowest $h = 253$). Children: Arad (366), Fagaras (176), Oradea (380), Rimnicu Vilcea (193). Frontier: {Fagaras:176, Rimnicu Vilcea:193, Timisoara:329, Zerind:374, Arad:366, Oradea:380}.

Step 3. **Expand Fagaras** (lowest $h = 176$). Child: Bucharest ($h = 0$). Frontier: {Bucharest:0, . . . }.

Step 4. **Expand Bucharest.** Goal found! Path: Arad → Sibiu → Fagaras → Bucharest. Total road cost: $140 + 99 + 211 = \mathbf{450\,km}$.

This is *not* the optimal path. The optimal route via Rimnicu Vilcea and Pitesti costs only 418 km. GBFS took the path that *looked* closest at each step but ended up longer overall.

The five-node graph example (GBFS)

The slide also shows a five-node graph. Nodes 1–5, with heuristic values $h(1) = 60$, $h(2) = 120$, $h(3) = 30$, $h(4) = 40$, $h(5) = 0$. Start: node 1, goal: node 5. Edge costs: $1 \to 2$: 100, $1 \to 3$: 100, $1 \to 4$: 100, $1 \to 5$: 75, $3 \to 5$: 125, $4 \to 5$: 50.

✎ **Worked example: GBFS on the five-node graph**

Step 1. **Expand node 1.** Frontier (sorted by h): {3:30, 4:40, 1(=2?)}. Actually frontier after expanding 1: nodes 2 ($h = 120$), 3 ($h = 30$), 4 ($h = 40$).

Step 2. **Expand node 3** (lowest $h = 30$). Child: node 5 ($h = 0$). Frontier: {5:0, 4:40, 2:120}.

Step 3. **Expand node 5.** Goal found! Path: $1 \to 3 \to 5$. Road cost: $100 + 125 = \mathbf{225}$.

GBFS expanded 2 nodes, generated 6, and found the path $1 \to 3 \to 5$ with cost 225. The optimal path $1 \to 5$ exists with edge cost 75, but GBFS missed it because node 5 was not a direct successor listed first. (Note: the slide shows the optimal to be 1-5 with cost 75, but GBFS chose $1 \to 3$ since $h(3) = 30 < h(5 - \text{direct})$ was not expanded first.)

✗ **Watch out**

GBFS ignores the cost already paid. It can pick an expensive route to a node that looks close, while a cheap direct route to the goal is skipped. Greediness is fast but not safe.

Where this shows up in ML. Greedy decoding in language models is the direct analogue: at each step, pick the token with the highest probability (lowest heuristic cost to a complete sentence). It is fast but often sub-optimal; beam search is the informed upgrade.

✓ **Key takeaway**

GBFS picks the node with the smallest $h(n)$ (closest-looking to goal). It is fast but not optimal and not complete. A greedy choice can lead down a long or dead-end path.

8 A* Search

A* fixes the flaw of GBFS. GBFS ignores the cost already paid ($g(n)$). UCS ignores the estimated cost remaining ($h(n)$). A* uses both. Its evaluation function is

$$f(n) = g(n) + h(n),$$

where:

- $g(n)$ is the actual cost from the start to node n,
- $h(n)$ is the heuristic estimate from n to the goal,
- $f(n)$ is the estimated total cost of the cheapest path through n.

A* extends the path that minimises $f(n)$. It is a best first search, like GBFS, but with a smarter evaluation.

☞ The intuition

A* balances two questions at once. "How much have I already spent?" and "How much do I think I have left?" A node that is cheap to reach but far from the goal scores the same as one that is expensive to reach but very close. A* finds the sweet spot.

★ Everyday picture

You are planning a road trip. The sat-nav shows you two options. Route A: you have driven 200 km and the remaining straight-line distance is 50 km. Route B: you have driven 100 km and the remaining straight-line distance is 200 km. Route A has a lower $f = 200 + 50 = 250$ vs. $f = 100 + 200 = 300$, so the sat-nav prefers Route A. A* does the same calculation for every frontier node.

A* on the five-node graph

Same graph as before. Edge costs: $1 \to 2$: 70, $1 \to 3$: 125, $1 \to 4$: 100, $4 \to 5$: 50. Heuristics: $h(1) = 60$, $h(2) = 120$, $h(3) = 70$, $h(4) = 40$, $h(5) = 0$.

✎ Worked example: A* on the five-node graph

Step 1. **Start at node 1.** $f(1) = g(1) + h(1) = 0 + 60 = 60$. Expand node 1. Generate neighbours: node 2: $g = 70$, $f = 70 + 120 = 190$; node 3: $g = 125$, $f = 125 + 70 = 195$; node 4: $g = 100$, $f = 100 + 40 = 140$. Frontier: {4:140, 2:190, 3:195}.

Step 2. **Expand node 4** (lowest $f = 140$). Generate node 5: $g = 100 + 50 = 150$, $f = 150 + 0 = 150$. Frontier: {5:150, 2:190, 3:195}.

Step 3. **Expand node 5** (lowest $f = 150$). Goal reached! Path: $1 \to 4 \to 5$. Total cost $g(5) = \mathbf{150}$.

A* found the optimal path $1 \to 4 \to 5$ with cost 150. Compare GBFS which found $1 \to 3 \to 5$ with cost 225. A* expanded only 2 nodes and generated 5.

A* on the Romania map (Example 2)

Start: Arad; Goal: Bucharest. $h(n)$ = straight-line distance to Bucharest. Key h values: Arad 366, Sibiu 253, Timisoara 329, Zerind 374, Rimnicu Vilcea 193, Fagaras 176, Pitesti 100, Bucharest 0.

✎ Worked example: A* Arad to Bucharest (slide trace)

Step 1. **Expand Arad.** $f(\text{Arad}) = 0 + 366 = 366$. Children: Sibiu $f = 140 + 253 = 393$; Timisoara $f = 118 + 329 = 447$; Zerind $f = 75 + 374 = 449$.

Step 2. **Expand Sibiu** (lowest $f = 393$). Children: Arad $f = 280 + 366 = 646$; Fagaras $f = 239 + 176 = 415$; Oradea $f = 291 + 380 = 671$; Rimnicu Vilcea $f = 220 + 193 = 413$. Frontier (relevant): {Rimnicu Vilcea:413, Fagaras:415, Timisoara:447, Zerind:449, ...}.

Step 3. **Expand Rimnicu Vilcea** (lowest $f = 413$). Children: Craiova $f = 366 + 160 = 526$; Pitesti $f = 317 + 100 = 417$; Sibiu $f = 300 + 253 = 553$. Frontier: {Fagaras:415, Pitesti:417, Timisoara:447,...}.

Step 4. **Expand Fagaras** (lowest $f = 415$). Child: Bucharest $f = 450 + 0 = 450$. Added

to frontier. Frontier: {Pitesti:417, Bucharest:450, ...}.

Step 5. **Expand Pitesti** (lowest $f = 417$). Child: Bucharest $f = 418 + 0 = 418$. This replaces the 450 entry. Frontier: {Bucharest:418, ...}.

Step 6. **Expand Bucharest** (lowest $f = 418$). Goal found! Path: Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest. Total cost $g = \mathbf{418\,km}$.

This is the true optimal path. Note how A* found a cheaper Bucharest entry (418) by going via Pitesti, overriding the 450 entry that came via Fagaras.

✗ Watch out

A* only guarantees optimality when the heuristic never overestimates. An overestimating heuristic inflates $f(n)$ for cheap nodes. Those nodes are then expanded too late. A* may return a sub-optimal path.

Where this shows up in ML. A* is used in **beam search**, a bounded-width variant. It also appears in neural code generation, which searches over program ASTs. MCTS variants that add a heuristic value network are another application. The $f = g + h$ idea underpins many policy-plus-value combinations in modern planning agents.

✓ Key takeaway

A* uses $f(n) = g(n) + h(n)$: cost so far plus estimated cost remaining. It expands nodes in order of f. With an admissible heuristic it is complete and optimal.

9 Admissibility and Consistency

A* is optimal only when the heuristic is "honest." There are two precise ways to define that honesty: *admissibility* and *consistency*.

9.1 Admissibility: never overestimate

A heuristic $h(n)$ is **admissible** (also called *optimistic*) if it never overestimates the true cost to reach the goal from n:

$$h(n) \;\leq\; h^*(n),$$

where $h^*(n)$ is the *true* (actual optimal) cost from n to the goal. An admissible heuristic may underestimate, but it must not exaggerate.

☞ The intuition

"Admissible" means the heuristic is an optimist. It says "at best, the goal is this far." Because it never overestimates, A* will not discard a node thinking it is too expensive when it is actually on the optimal path.

★ Everyday picture

The straight-line distance between two cities is admissible for a road-route planner. The actual road must be at least as long as the straight-line distance (roads are not shorter

than a crow's flight). So the straight-line distance never overestimates the road cost.

9.2 Consistency: smooth estimates

A heuristic $h(n)$ is **consistent** (also called *monotone*) if, for every node n and every successor n' of n reached by action a,

$$h(n) \;\leq\; c(n, a, n') + h(n'),$$

where $c(n, a, n')$ is the cost of moving from n to n' via action a. In words: the estimated remaining cost from n does not decrease by more than the actual step cost when you move to n'.

☞ The intuition

Consistency is the triangle inequality for heuristics. The estimate at n should be no more than one step cost plus the estimate at the destination. If the estimate drops by more than the step cost, A* may skip n too early. It would then need to re-open n later, which is slow. Consistency prevents that.

★ Everyday picture

Imagine a sat-nav that tells you "50 km remaining" at one junction and "5 km remaining" at the very next junction 2 km away. That is a 45 km drop for a 2 km step. The estimate is inconsistent. A good sat-nav's remaining-distance estimate decreases by at most the distance you actually drove — it does not suddenly jump.

Why these properties matter for A*

- If h is **admissible**, A* on a tree is optimal.
- If h is **consistent**, $f(n)$ values are non-decreasing along any path ($f(n') \geq f(n)$). This means once A* expands a node, it has found the optimal path to that node. No need to re-open it.
- Every consistent heuristic is admissible (when $h(\text{goal}) = 0$). But an admissible heuristic need not be consistent.

Checking admissibility and consistency: the five-node example

The slide checks a heuristic for the five-node graph with goal = node 3. Given: $h(1) = 80$, $h(2) = 60$, $h(3) = 0$, $h(4) = 200$, $h(5) = 190$.

✎ Worked example: Admissibility and consistency check

We check each node. The *true* costs $h^*(n)$ are the shortest actual path costs from each node to node 3.

Step 1. **Node 1.** True shortest cost to node 3 via $1 \to 3$: 125 (assuming edge $1 \to 3 = 125$ from the graph). $h(1) = 80 \leq 125$: **admissible (Y)**. Consistency check arc $5 \to 1$: $h(5) = 190 \leq c(5, 1) + h(1) = 190 + 80 = 270$? Wait — check arc $5 \to 1$: is $190 \leq g(5, 1) + 80$? The slide notes arc $5 \to 1$: $190 \leq 155$ is **false**. Node 1 is **not consistent (N)**.

Step 2. **Node 2.** $h(2) = 60$. True cost: e.g. via $2 \to 3$: 75. $60 \leq 75$: would be admissible. But the slide shows $h(2) = 60 > h^*(2)$, so **not admissible (N)**. Consistent checks pass.

Step 3. **Node 3.** $h(3) = 0$. At the goal so $h^*(3) = 0$. $0 \leq 0$: **admissible (Y)**.

Step 4. **Node 4.** $h(4) = 200$. True cost: e.g. $4 \to 3 = 100$. $200 > 100$: **not admissible (N)**. Slide shows it admissible with $h(4) = 200 \leq h^*(4)$ and consistency checks pass.

Step 5. **Node 5.** $h(5) = 190$. Consistency arc $2 \to 5$ and $4 \to 5$ both hold. **Consistent (Y)**.

The key lesson: a heuristic that is *not* admissible can cause A* to return a sub-optimal solution. A heuristic that is not *consistent* can force re-expansion of nodes.

✗ Watch out

Admissibility is required; consistency is stronger and more convenient. Never use an overestimating heuristic with A* if you need the optimal solution. An overestimating heuristic can prune the optimal path before A* ever finds it.

9.3 What makes a good heuristic?

Three properties make a heuristic good for A*:

1. **Admissible:** never overestimate the true cost.
2. **Consistent:** satisfy the triangle inequality across edges.
3. **Informative:** be as close to the true cost as possible.

A more informative heuristic means A* explores fewer nodes. If two admissible heuristics h_1 and h_2 satisfy $h_2(n) \geq h_1(n)$ for all n, then h_2 *dominates* h_1 and A* with h_2 expands fewer (or equal) nodes.

$$h_2(n) \geq h_1(n) \quad \Rightarrow \quad \text{A* with } h_2 \text{ expands fewer nodes.}$$

Heuristic type	Effect on A*
$h(n) = 0$	Always optimal, but slow. Same as UCS.
Admissible but weak	Optimal, but may explore many nodes.
Admissible and informative	Optimal and efficient.
Overestimating	Faster, but loses optimality guarantee.
Inconsistent	May require re-opening already-expanded nodes.

Where this shows up in ML. Admissibility is the exact condition that makes alpha–beta pruning in game trees safe. Value functions learned by reinforcement learning act as heuristics; their accuracy determines how efficiently the planner searches. In theorem provers and code synthesisers, a learned admissible heuristic can cut search time by orders of magnitude.

✓ Key takeaway

An admissible heuristic never overestimates: $h(n) \leq h^*(n)$. A consistent heuristic also satisfies the triangle inequality. With an admissible heuristic, A* is complete and optimal. A more informative heuristic makes A* faster.

Glossary & symbols

Key terms.

BFS Breadth First Search: explores nodes level by level using a queue. Complete, optimal for uniform costs.

DFS Depth First Search: follows one path to its end using a stack before backtracking. Not optimal; low memory.

UCS Uniform Cost Search: expands the node with the lowest $g(n)$. Complete and optimal for non-negative costs.

IDS Iterative Deepening Search: runs DFS with increasing depth limits. Combines BFS optimality with DFS memory.

heuristic $h(n)$ A function that estimates the cost from node n to the goal. Used by informed search algorithms.

GBFS Greedy Best First Search: picks the node with smallest $h(n)$. Fast but not optimal and not complete.

A* Expands the node with smallest $f(n) = g(n) + h(n)$. Complete and optimal when the heuristic is admissible.

admissible A heuristic property: $h(n) \leq h^*(n)$. It never overestimates the true remaining cost.

consistent A heuristic property: $h(n) \leq c(n, a, n') + h(n')$ for all edges. Also called monotone. Implies admissible.

frontier The set of nodes generated but not yet expanded; managed as a queue, stack, or priority queue depending on the algorithm.

branching factor b
The average number of children per node in the search tree.

Symbols at a glance.

Symbol	Meaning
b	branching factor: average children per node
d	depth of the shallowest goal node
m	maximum depth of any node in the search tree
$g(n)$	actual cost from the start node to node n
$h(n)$	heuristic estimate of cost from n to the goal
$h^*(n)$	true (optimal) cost from n to the goal
$f(n)$	A* evaluation function: $f(n) = g(n) + h(n)$
$c(n, a, n')$	cost of action a that moves from node n to n'